

National University of Computer and Emerging Sciences

Karachi Campus

Final Exam Paper Part-B

CLO #1: To apply acquired knowledge to solve computing problems complexities and proofs

Question 3 [5 Marks]: Briefly answer the following questions

Total Time (Hrs): 3
Total Marks: 50
Total Questions: 8

Design and Analysis of Algorithms (CS2009)

Date: December 30, 2025

Course Instructor(s)

Dr. Nasir Uddin, Dr.
Kamran, Dr. Fahad
Sherwani, Mr. Sandesh
Kumar, Mr. Kashif, Mr.
Zulfiqar, Ms. Fizza

- a) **[1 Mark]** In Boyer-Moore, why do we compare the pattern with the text from right to left instead of left to right? Give one advantage.
- b) **[1 Mark]** What is the worst-case time complexity of the Boyer-Moore algorithm? Give an example pattern and text that leads to this worst-case behavior.
- c) **[1 Mark]** For pattern $P = \text{"TESTING"}$ and Text $T = \text{"TEXTING TESTING"}$
- Construct the bad character shift table (last occurrence table) for the standard Boyer-Moore version
 - Using this table, compute the shift when matching against text "TEXTING TESTING" and a mismatch occurs:
- d) **[2 Marks]** Pattern: $P = \text{"ACABAC"}$ and Text: $T = \text{"BACAACABACAD"}$ Apply Boyer-Moore algorithm (Good Suffix Rule only) starting at alignment position $s = 0$. Show the steps until first full match.

Solution:

- a) **In Boyer-Moore, why do we compare the pattern with the text from right to left instead of left to right? Give one advantage.**

Solution:

When a mismatch happens, the bad character rule can shift the pattern by more than one position (Characters), because we can align the mismatched text character with its rightmost occurrence in the pattern (or shift past it entirely). Right-to-left comparison maximizes the chance of early mismatch detection, leading to faster skipping over the text.

- b) **What is the worst-case time complexity of the Boyer-Moore algorithm? Give an example pattern and text that leads to this worst-case behavior.**

Solution:

The Boyer-Moore algorithm has a worst-case time complexity of $O(nm)$, where:

n is the length of the text

m is the length of the pattern

This occurs when both rules (bad character and good suffix) provide minimal shifts, often just one position per mismatch.

- c) For pattern $P = \text{"TESTING"}$ and Text $T = \text{"TEXTING TESTING"}$
- Construct the bad character shift table (last occurrence table) for the standard Boyer-Moore version (consider alphabet = $\{A...Z\}$).
 - Using this table, compute the shift when matching against text "TEXTING TESTING" and a mismatch occurs:

National University of Computer and Emerging Sciences

Karachi Campus

Solution:

Bad character shift table

Shift Value Formula: $\text{Max}(1, P.\text{Length} - \text{Index} - 1)$

Indices	0	1	2	3	4	5	6	
Characters	T	E	S	T	I	N	G	
Shift Value	-----	5	4	3	2	1	1	

Compute the shift when matching against text "TEXTING TESTING".

Left Align the Pattern with Text and start matching from right most

Text	T	E	X	T	I	N	G		T	E	S	T	I	N	G
Pattern	T	E	S	T	I	N	G								
			Not	Match	Match	Match	Match								

Here Mismatch occurs at X - Text[2] and Pattern[2], here Shift Values is **S** which is **4** in the **Shift Table**.

Updated Align will be like this .

Text	T	E	X	T	I	N	G		T	E	S	T	I	N	G
Pattern					T	E	S	T	I	N	G				

d)

Pattern: P = "ACABAC" (length m = 6)

Text: T = "BACAACABACAD"

Start: Alignment position s = 0

Step 1: Alignment s=0

Compare right to left:

B	A	C	A	A	C	A	B	A	C	A	D
A	C	A	B	A	C						
			Not	Match	Match						
			Match								

Matched suffix length = 2 ("AC")

Good Suffix Rule: For suffix length 2 in P="ACABAC", earlier "AC" at start (positions 0–1) with preceding char different → shift = 4

New S = 0 + 4 = 4

B	A	C	A	A	C	A	B	A	C	A	D
				A	C	A	B	A	C		
				Match	Match	Match	Match	Match	Match		

Step 2: Alignment s=4

T[4..9] = A A C A B A

Right to left:

National University of Computer and Emerging Sciences

Karachi Campus

All Pattern Match

CLO #4: To construct and analyze real world problems solutions using different algorithms design techniques. i.e. Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms.

Question 4 [5 Marks]: Suppose you are given an array A with n entries, with each entry holding a distinct number. You are told that the sequence of values $A[1], A[2], \dots, A[n]$ is unimodal. For some index p between 1 and n , the values in the array entries increase up to position p in A and then decrease the remainder of the way until position n . (So if you were to draw a plot with the array position j on the x-axis and the value of the entry $A[j]$ on the y-axis, the plotted points would rise until x-value p , where they'd achieve their maximum, and then fall from there on.) Design the algorithm where You'd like to find the "peak entry" p without having to read the entire array in fact, by reading as few entries of A as possible. Show how to find the entry p by reading at most $O(\log n)$ entries of A and also provide the correctness of the algorithm using loop invariant.

Solution:

Input Array $A[1..n]$, unimodal with distinct elements

Output Index p such that $A[p]$ is the maximum

low = 1

high = n

while low < high:

 let mid = $\lfloor (\text{low} + \text{high}) / 2 \rfloor$

 read $A[\text{mid}]$ and $A[\text{mid} + 1]$

 if $A[\text{mid}] < A[\text{mid} + 1]$:

 The peak is to the **right**

 Set low = mid + 1

 else

 The peak is to the **left or at mid**

 Set high = mid

low == high == p

Return p

Number of Iterations = $O(\log n)$

Total array reads = $O(\log n)$

We prove that the algorithm always maintains the invariant:

The peak index p lies in the interval $[\text{low}, \text{high}]$.

- Initially, $[1, n]$ clearly contains p .
- At each iteration:
 - If $A[\text{mid}] < A[\text{mid}+1]$, the array is increasing at mid, so the peak cannot be at or left of mid; thus $p \in [\text{mid}+1, \text{high}]$.
 - If $A[\text{mid}] > A[\text{mid}+1]$, the array is decreasing at mid, so the peak must be at or left of mid; thus $p \in [\text{low}, \text{mid}]$.
- In both cases, the invariant is preserved and the interval size shrinks.

When low = high, only one index remains, which must be the peak.

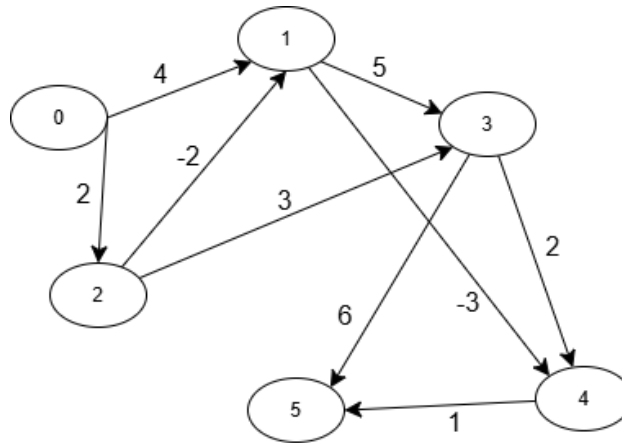
CLO #3: To evaluate generic algorithmic solutions such as sorting, searching and graphs applied to real-world problems

Question 5 [5 Marks]: You are provided the following directed graph and your task is to analyze it to achieve the following tasks.

National University of Computer and Emerging Sciences

Karachi Campus

- a) **[1.25 Marks]** To determine a suitable and efficient algorithm that you learned in class find out the shortest paths from vertex 0 to all other vertices. Justify your choice by explaining why the other algorithm cannot be used.?
- b) **[2 Marks]** Using the algorithm you selected, find the shortest distance from source vertex 0 to all other vertices (1, 2, 3, 4, 5). The relaxation of the edges should be done strictly using the following order: (0→1), (0→2), (2→1), (2→3), (1→3), (1, 4), (3→4), (3→5), (4→5)
- c) **[1.75 Marks]** Use your algorithm to determine if there is any negative cycle in the graph. Don't use brute force way to determine it.



Solution:

a) Algorithm Selection

Selected Algorithm: Bellman-Ford

Justification: The graph contains a **negative edge weight** (-2 from vertex 2 to vertex 1, and -3 from vertex 3 to vertex 4).

- **Dijkstra's algorithm cannot be used** because it assumes all edge weights are non-negative. It uses a greedy approach that may fail with negative weights, as a shorter path might be found later through negative edges.
- **Bellman-Ford algorithm is appropriate** because it can handle negative edge weights and will correctly compute shortest paths (it also detects negative cycles if present).

b) Bellman-Ford Algorithm Steps

Initial Setup:

- Source: vertex 0
- Initial distances: $d[0]=0$, $d[1]=\infty$, $d[2]=\infty$, $d[3]=\infty$, $d[4]=\infty$, $d[5]=\infty$

Iteration 1:

- Relax (0→1): $d[1] = \min(\infty, 0+4) = 4$

National University of Computer and Emerging Sciences

Karachi Campus

- Relax (0→2): $d[2] = \min(\infty, 0+2) = 2$
- Relax (2→1): $d[1] = \min(4, 2+(-2)) = 0$
- Relax (2→3): $d[3] = \min(\infty, 2+3) = 5$
- Relax (1→3): $d[3] = \min(5, 0+5) = 5$
- Relax (3→4): $d[4] = \min(\infty, 5+2) = 7$
- Relax (3→5): $d[5] = \min(\infty, 5+6) = 11$

Iteration 2:

- Relax (3→1): $d[1] = \min(0, 5+(-3)) = 0$
- Relax (4→5): $d[5] = \min(11, 7+1) = 8$

Iterations 3-5: No further updates occur.

Final Shortest Distances from vertex 0:

- $d[0] = 0$
- $d[1] = 0$
- $d[2] = 2$
- $d[3] = 5$
- $d[4] = 7$
- $d[5] = 8$

C) Negative Cycle Detection (1.75 marks)

a) Does this graph contain a **negative weight cycle**?

Answer: NO, this graph has not contains a negative weight cycle.

b) Explain the method used by your chosen algorithm to detect negative cycles.

Bellman-Ford's Negative Cycle Detection Method:

The Bellman-Ford algorithm detects negative cycles using the following principle:

- For a graph with V vertices, the shortest path between any two vertices contains at most **(V-1) edges** (if no negative cycles exist).
- The algorithm performs **(V-1) iterations** to find all shortest paths.
- **After (V-1) iterations**, if we can still relax any edge (i.e., find a shorter path), then a **negative cycle exists** that is reachable from the source.

National University of Computer and Emerging Sciences

Karachi Campus

Detection Process:

1. Run (V-1) relaxation iterations to compute shortest paths
2. Perform one additional iteration (the Vth iteration)
3. If any distance can be reduced in this additional iteration, a negative cycle exists

CLO#3: To evaluate geometric algorithmic solutions applied to solve real-life problems

Question 6 [5 Marks]: FAST University Karachi plans to install high-speed secure cables to connect security cameras deployed at various locations across the campus. Due to cost constraints, only the cameras lying on the outer boundary of the campus are to be connected, such that all other cameras remain enclosed.

Each camera location is represented as a point in a 2-D plane. Given the following camera locations:

$P = \{(1,1), (2,3), (4,4), (6,2), (5,1), (3,2), (4,2)\}$

- a) **[3.5 Marks]** Apply the Graham Scan algorithm to compute the convex hull of the given points. Clearly show:
Selection of pivot point (starting point), sorting based on polar angle and point inclusion or exclusion decisions using orientation test.
- b) **[1.5 Marks]** Identify the points (if there is any) that were not included because eliminated them for being collinear

CLO #4: To construct and analyze real world problems solutions using different algorithms design techniques i.e. Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms.

Question 7 [5 Marks]: A furniture manufacturing company named Interwood produces wooden beams of fixed length 10 meters. The company can sell the beams either as whole units or by cutting them into smaller lengths. The market prices for each possible length are shown below:

Length (m)	1	2	3	4	5	6	7	8	9	10
Price (\$)	2	6	7	10	13	17	17	20	24	30

- a) **[2 Marks]** Identify the optimal substructure property and define the dynamic programming formulation and recurrence relation
- b) **[3 Marks]** Perform the bottom-up dry run and fill the table from length 1 to 10 and determine the maximum obtainable revenue and the corresponding cuts and write down its worst case complexity

Solution:

- (a) Identify the optimal substructure property and define the dynamic programming formulation and recurrence relation

Best revenue for length n depends on best revenues of smaller lengths.

Let

$dp[n]$ = maximum revenue for rod of length n

Base Case

$dp[0] = 0$

$dp[n] = 1 \leq i \leq n \max(price[i] + dp[n-i])$

National University of Computer and Emerging Sciences

Karachi Campus

- (b) Perform the bottom-up dry run and fill the table from length 1 to 10 and determine the maximum obtainable revenue and the corresponding cuts and write down its worst case complexity

Length	dp[n]	Best Cut
1	2	1
2	6	2
3	8	1+2
4	12	2+2
5	14	2+3
6	18	3+3
7	20	1+6
8	24	2+6
9	26	3+6
10	30	no cut

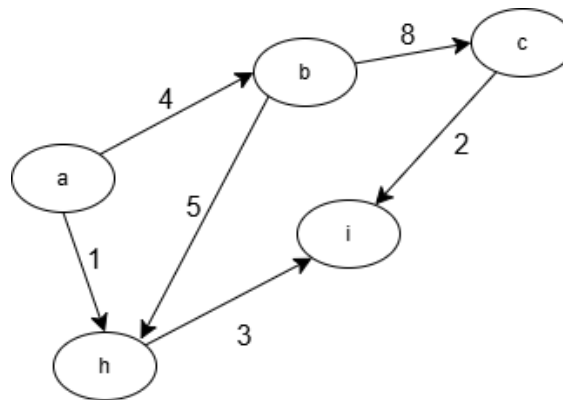
Maximum Revenue = 30

Time Complexity: $O(n^2)$

CLO#3 To evaluate geometric algorithmic solutions applied to solve real-life problems

Question 8 [5 Marks]: Briefly answer the following questions

- a) **[2 Marks]** Compute the minimum spanning tree of the graph given below using either Prim's and Kruskal's algorithm. Briefly explain your solution.



- b) **[3 Marks]** Below is the most straight forward Pseudocode/code to implement Prim's algorithm using adjacency matrix representation of the graph to find the minimum spanning trees. Discuss the time complexity of the algorithm given below, and also briefly explain how you can improve the time complexity of the given Prim's algorithm using a different representation of the graph.

Solution:

- a) We can't compute a minimum spanning tree for a directed graph. An MST is a subset of edges in an undirected graph that connects all vertices together with the minimum possible total edge weight, without cycles. In a directed graph, edges have orientation. If you pick edges to connect all vertices, you might be able to reach some nodes but not return (like node 'i'), so the notion of a spanning "tree" doesn't apply directly.
- b) The total time complexity of the given implementation of Prim's algorithm that uses an adjacency matrix to represent the graph is: $O(V^2)$. Outer loop runs $V - 1$ times, where V is the number of vertices.

National University of Computer and Emerging Sciences

Karachi Campus

Finding the minimum key vertex not yet included in MST takes $O(V)$ time in each iteration. Inner loop iterates over all V vertices to update adjacent vertices, taking $O(V)$ time.

To improve the efficiency, we can use an adjacency list representation along with a min-priority queue (heap). This allows us to efficiently extract the minimum key vertex in $O(\log V)$ time, and only iterate over adjacent vertices (not all V), thus improving the overall time complexity to $O((V+E)\log V)$.

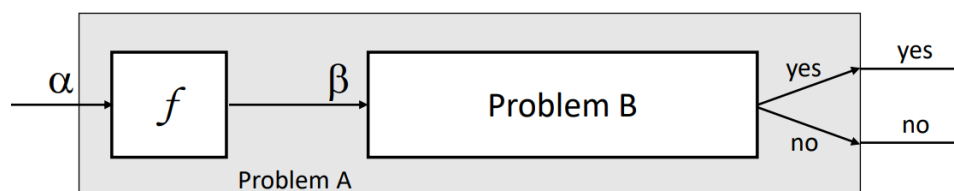
CLO #1: To apply acquired knowledge to solve computing problems complexities and proofs

Question 9 [10 Marks]: Briefly answer the following questions

- a) **[1.25 Marks]** What is 3-SAT problem? Describe its conjunctive normal form.
- b) **[1.25 Marks]** Assume that you developed a 2-approximation algorithm for an NPC problem. What does it mean if the problem is a maximization or minimization problem?
- c) **[1.25 Marks]** What is the difference between TSP and Metric TSP.
- d) **[1.25 Marks]** The expression $A \leq_p B$ for problem A and B represents what?
- e) **[1.25 Marks]** A graph G having $|V|$ vertices has a clique of size greater than k. Then what is the size of the independent set in the complement graph of G. Justify your answer.
- f) **[1.25 Marks]** Create a diagram representing the concept of reduction with respect to NPC problems.
- g) **[1.25 Marks]** A 3-SAT that is not satisfiable has 7 clauses then what should be the size of independent set if it is reduced to graph problem.

Solution:

- a) 3-SAT is a decision problem that asks whether a Boolean formula can be made true by some assignment of variables.
In conjunctive normal form (CNF), the formula is an AND of clauses, where each clause is an OR of exactly three literals.
Example: $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_4 \vee x_5)$
- b) **Minimization:** the algorithm's solution cost is at most $2 \times \text{OPT}$.
Maximization: the algorithm's solution value is at least $\text{OPT} / 2$.
- c) In general TSP edge weights are arbitrary; no restriction on distances whereas in Metric TSP edge weights satisfy the triangle inequality. Metric TSP allows approximation algorithms; general TSP does not.
- d) $A \leq_p B$ means problem A is polynomial-time reducible to problem B. Any instance of A can be transformed into an instance of B in polynomial time. Thus, B is at least as hard as A.
- e) If graph G has a clique of size greater than k, then its complement graph of G has an independent set of size greater than k. This is because cliques in G correspond exactly to independent sets in complement graph.
- f)



- g) If the 3-SAT instance has 7 clauses, the target independent set size is 7. Since the formula is unsatisfiable, no independent set of size 7 exists.